

Secure Web Infrastructure & WordPress Hardening

Proxmox • Linux • Docker • Nginx • WordPress

Project brief

BUSINESS CONTEXT

Marconi Law Firm required a controlled web-platform proof of concept that demonstrated traditional LAMP administration and a containerized application path.

The engagement emphasized secure configuration, validation evidence, and honest documentation of operational risk.

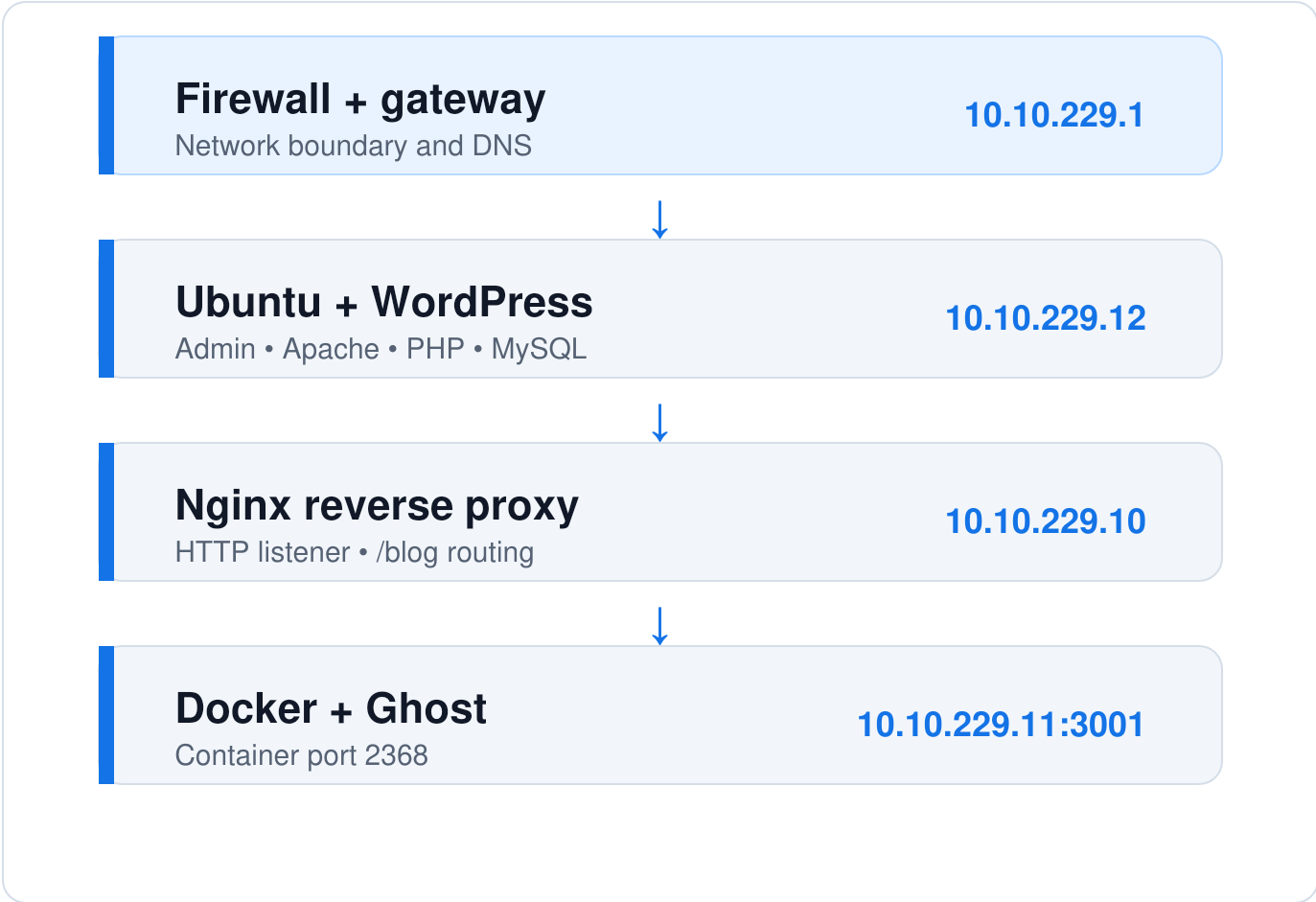
Infrastructure challenge

- Three Linux service roles
- Two application delivery patterns
- One private lab network

Engineering response

- Proxmox-based virtual environment
- Sanitized, repeatable configuration
- Evidence-backed validation and runbooks

Architecture and service boundaries



AUTHORITATIVE ADDRESS PLAN	
Gateway / firewall	10.10.229.1
Nginx reverse proxy	10.10.229.10
Docker + Ghost	10.10.229.11:3001
WordPress LAMP	10.10.229.12
Traffic is segmented by role: client-to-web, Nginx-to-Ghost, and localhost-only WordPress-to-MySQL.	

Implementation sequence

PHASE 1

Provision

Built the Proxmox network, assigned static addresses, set hostnames, and established SSH administration.

PHASE 2

Deploy

Installed Docker, Ghost, Nginx, Apache, MySQL, PHP, and WordPress across defined service roles.

PHASE 3

Harden + validate

Applied permissions, protected configuration, enabled security controls, tested services, and documented remediation.

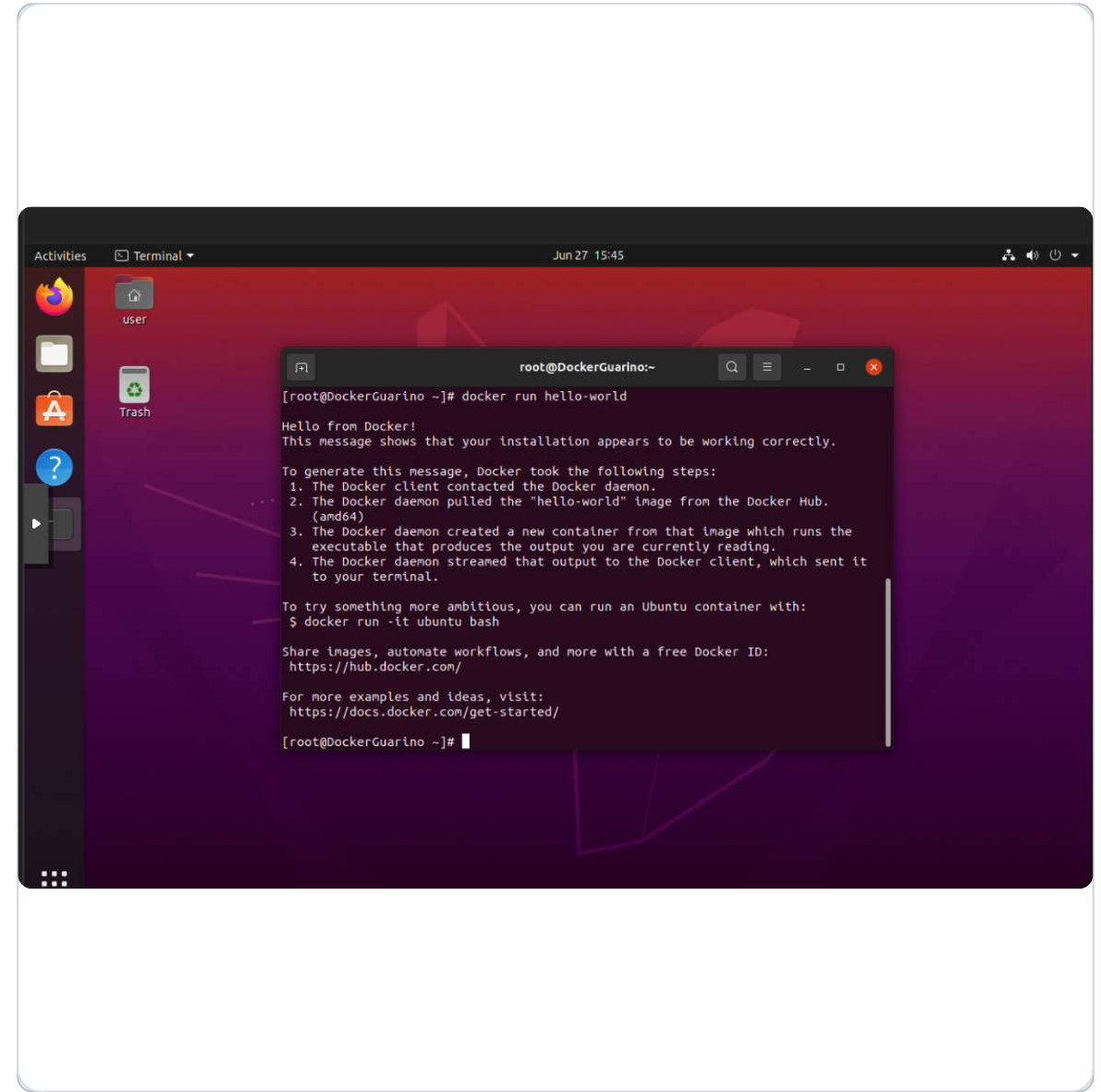
Docker platform validation

Validated container runtime

Docker CE was installed on Rocky Linux, enabled at boot, and tested with the standard hello-world container.

The Ghost workload was then defined with persistent storage and a host-to-container port mapping of 3001:2368.

VALIDATED



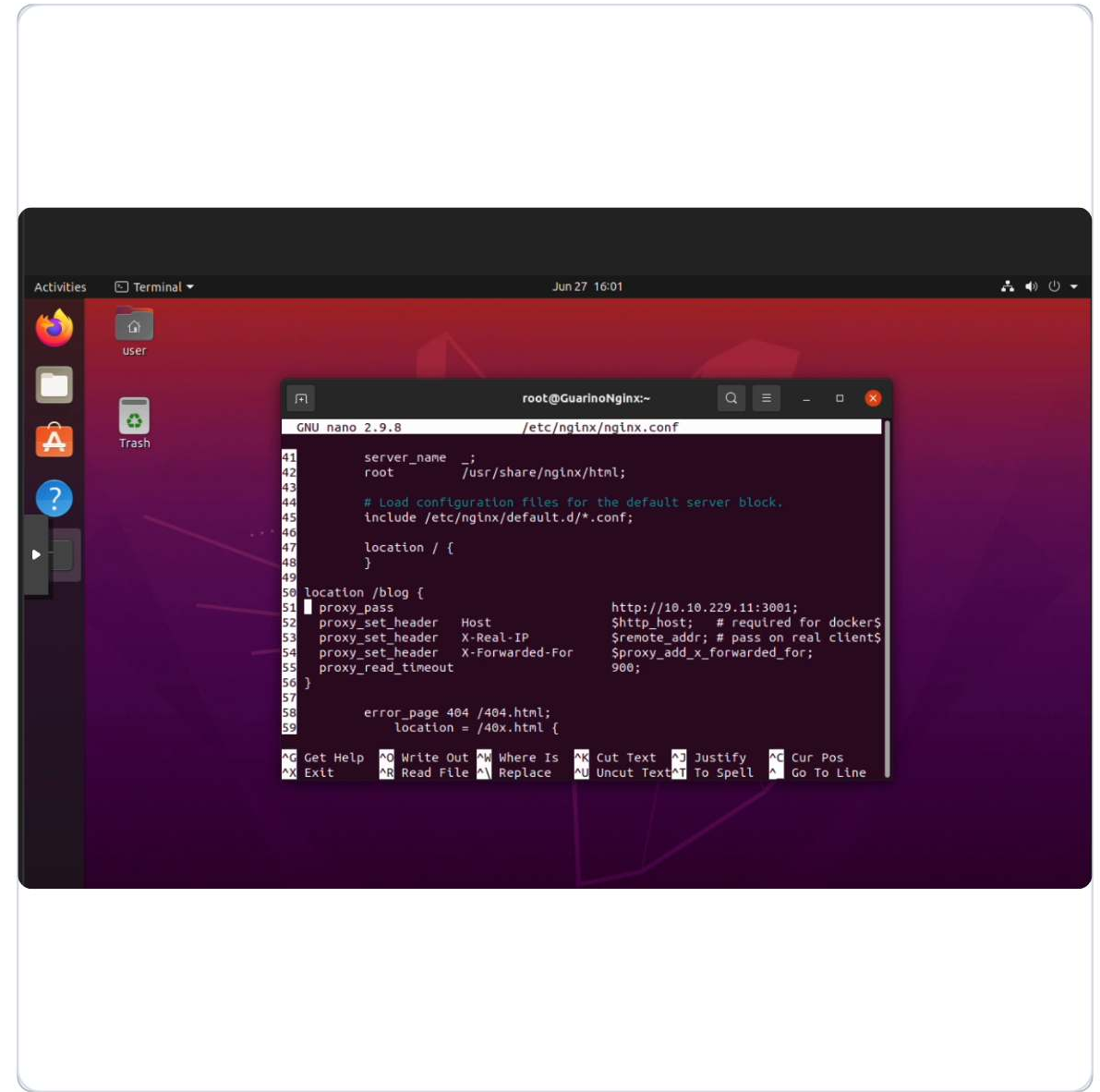
Nginx reverse proxy

Separated edge and application roles

Nginx runs on a dedicated Rocky Linux node and routes /blog traffic to the Docker host on TCP 3001.

The configuration forwards Host, X-Real-IP, and X-Forwarded-For headers to preserve request context for the upstream application.

CONFIGURATION
VALIDATED

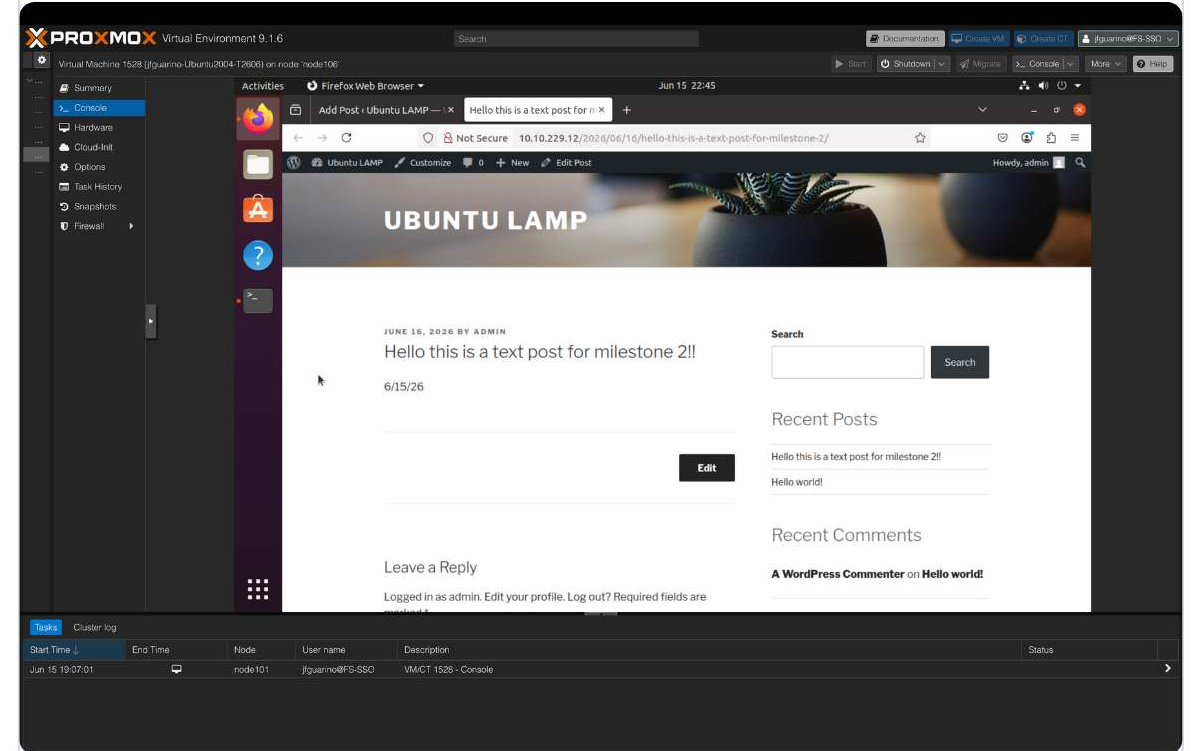


WordPress on Ubuntu LAMP

Delivered a functional web platform

Apache, MySQL, PHP, and WordPress were deployed on Ubuntu. URL rewriting was enabled, the database account was limited to localhost, and application content was validated in the browser.

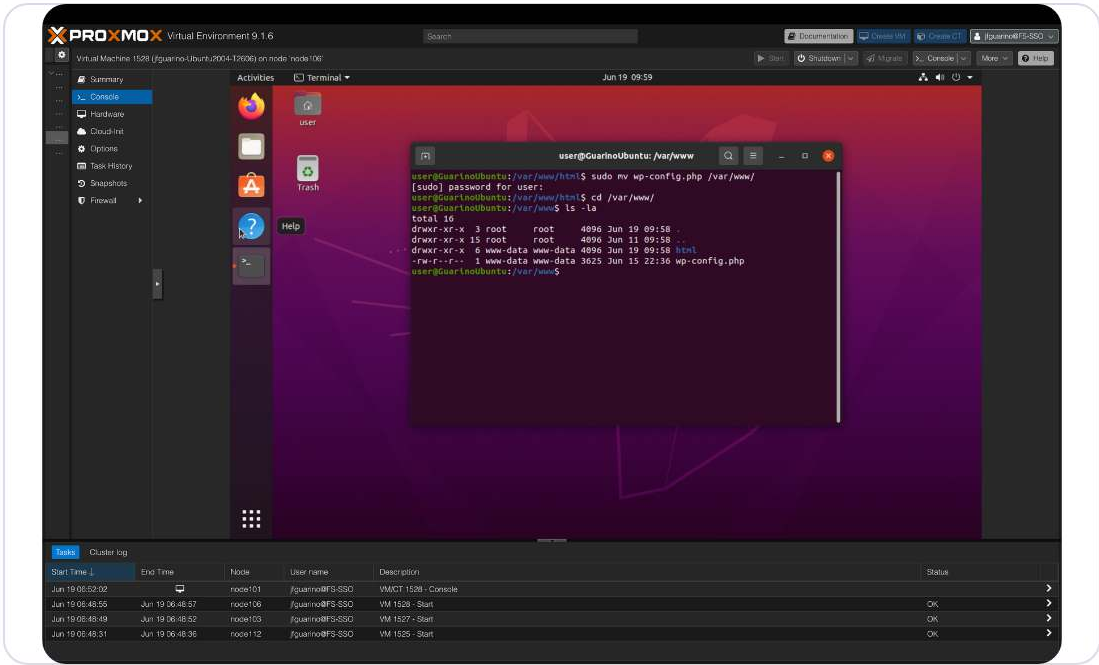
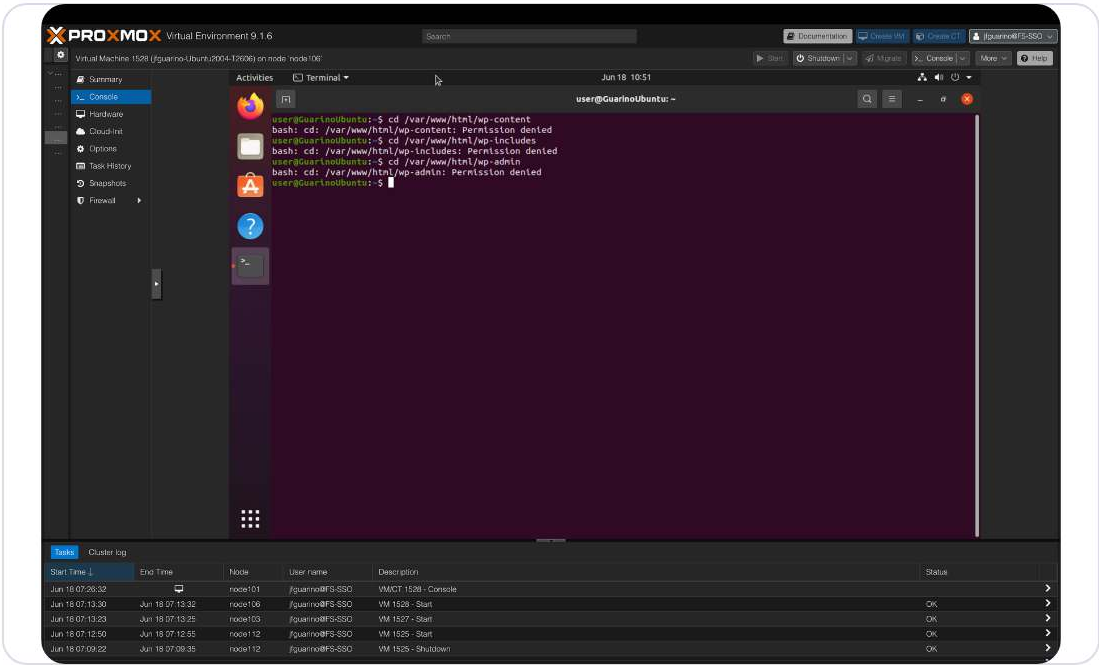
A test post confirmed end-to-end application delivery.



FUNCTIONAL

WordPress hardening controls

Controls were applied at the operating-system and application-configuration layers, then retested for expected access behavior.



Least-privilege permissions

Directories 755 • Files 644 • Controlled ownership

Protected configuration

wp-config.php moved above /var/www/html

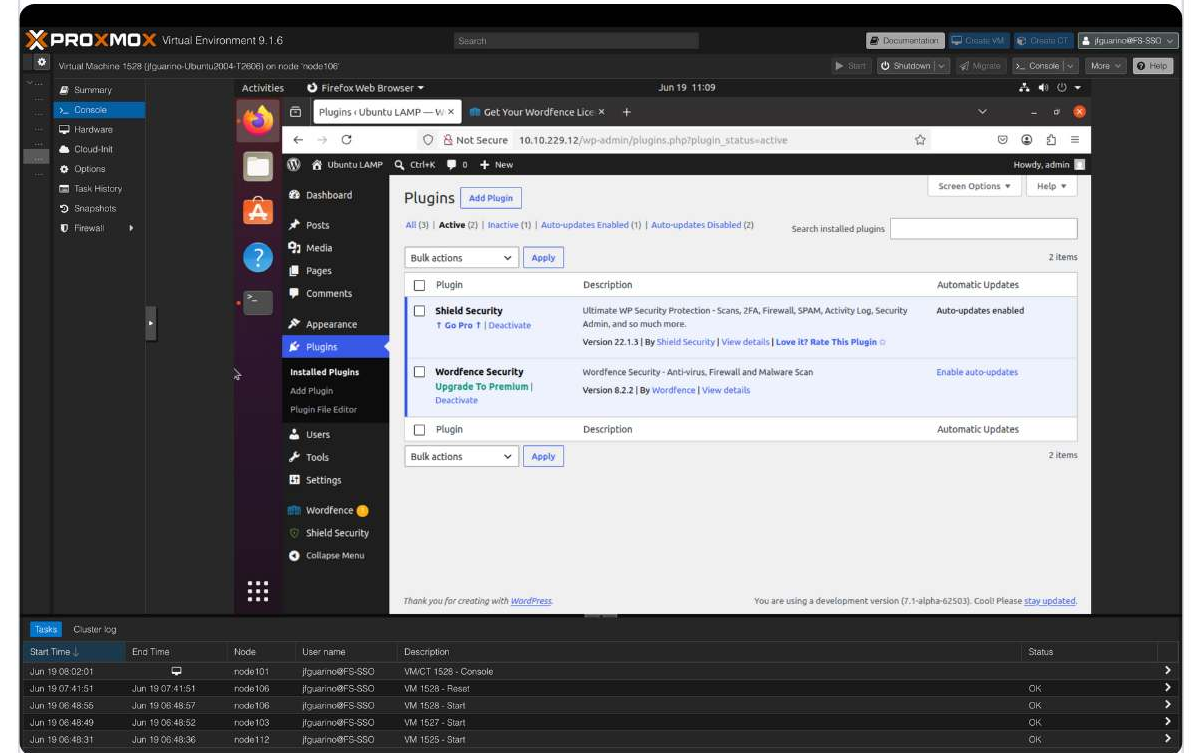
VALIDATED

Application-layer security

Defense in depth

Shield Security and Wordfence were activated to add application firewalling, scanning, monitoring, and login protection.

Production guidance recommends selecting one primary control after compatibility and resource-impact testing to avoid unnecessary overlap.

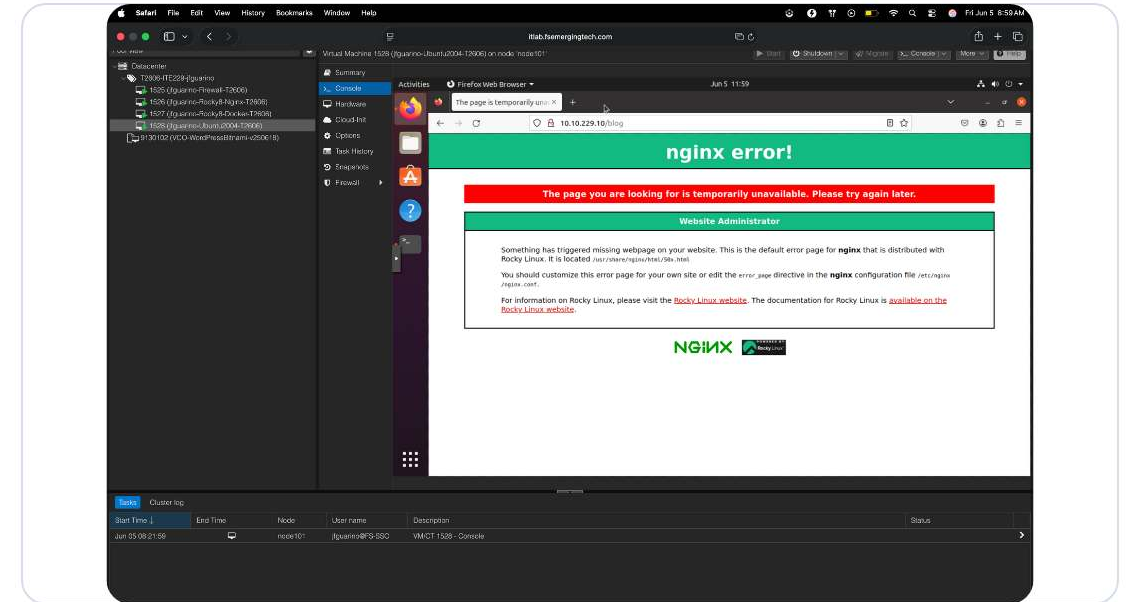
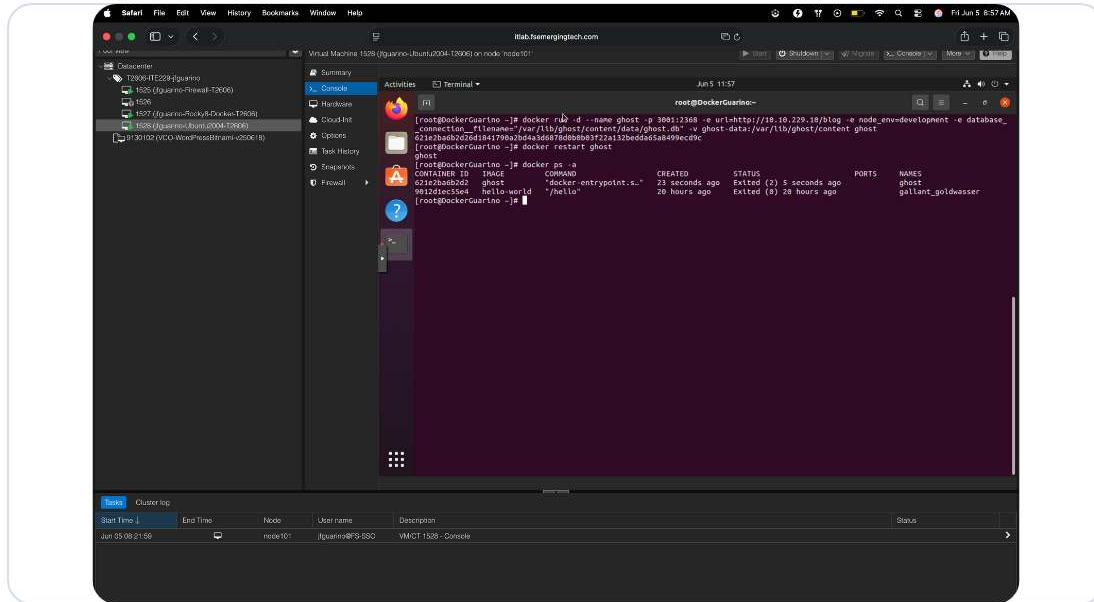


CONTROLS ACTIVE

Troubleshooting: Ghost upstream failure

REMEDATION REQUIRED

The service boundary was isolated correctly: Nginx remained available, while the Ghost container did not remain healthy.



REMEDATION PATH

- 1 Inspect container logs
- 2 Validate URL + SQLite path
- 3 Confirm 3001:2368 mapping
- 4 Test upstream directly
- 5 Retest /blog/

Skills demonstrated

ENGINEERING CAPABILITIES

The repository converts hands-on lab work into reviewable infrastructure documentation, sanitized configuration, validation evidence, and an actionable operations runbook.

Virtualization + Linux

Proxmox • Rocky Linux • Ubuntu • SSH • systemd

Containers + delivery

Docker • Compose • Nginx • Reverse proxy • TCP/IP

Web + data platform

Apache • MySQL • PHP • WordPress • Rewrite rules

Security + operations

Permissions • Secret hygiene • Validation • Troubleshooting

Infrastructure built. Evidence captured. Risk communicated.

Guarino IT Solutions

Linux • Infrastructure • Security

Portfolio project available on [GitHub](#)